Smart Farming Roadmap

Smart Farming (SIH 25099) — Project Overview & Implementation Roadmap

1. What the Project Actually Is

Strip away the buzzwords and this is a **4-stage AI pipeline wrapped in a web service**:

1. A farmer uploads a leaf photo (mobile/web).
2. **OpenCV** cleans and validates the image (is it usable? crop out the leaf).
3. A stack of **deep learning models** answer: *what crop, what disease, how bad, any pests?*
4. An **LLM** turns those raw predictions into a farmer-readable recommendation (fertilizer, pesticide, irrigation advice).

Everything else — FastAPI, PostgreSQL, MinIO, React, Docker — exists to move data reliably between these four stages and to store the results. The project is really two disciplines glued together:

- **Data Engineering** = getting a clean, validated, well-structured image (and its metadata) from "farmer's phone" to "model input tensor," and getting predictions from "model output" to "stored record."
- **Data Science** = building the models that sit inside that pipeline (crop ID → disease → severity → pests) and making sure they're accurate and versioned.

The key architectural idea from the discussion: every stage is an **independent, stateless function** that takes a shared JSON-like `request` object, adds its own fields to it, and passes it to the next stage. A single **pipeline orchestrator** calls these functions in order — no stage calls another stage directly. This is what makes the system look like microservices without the operational overhead of actually running microservices in a hackathon timeframe.

2. Where OpenCV vs. EfficientNet Fit

Role	Tool	Job
Image cleanup	OpenCV	Blur/brightness check, background removal, leaf cropping, CLAHE enhancement, denoising
Classification	EfficientNet (deep learning)	"What crop / what disease is this?"
Explainability	Grad-CAM + OpenCV	Heatmap overlay showing *why* the model predicted that

OpenCV never replaces the model — it's the preprocessing layer that makes the model's job easier and makes garbage input (blurry, cluttered, badly lit photos) get caught before it wastes a model call.

3. Build Order (Do It in This Sequence)

Building frontend-first is the most common mistake in hackathon projects — it leaves you demoing a UI with fake data at 2 a.m. Build bottom-up instead:

```
1. OpenCV Preprocessing Service       (standalone, testable on its own)
2. Crop Identification Service        (EfficientNet-B0)
```

```
3. Disease Classification Service     (EfficientNet-B2 / ConvNeXt, conditioned on crop)
4. Severity Estimation Service        (OpenCV segmentation + area calc)
5. Pest Detection Service             (YOLOv8, optional/stretch)
6. Recommendation Service             (LLM + weather API)
7. Pipeline Orchestrator              (glues 1–6 together)
8. FastAPI Endpoints                  (exposes orchestrator as /predict)
9. React Dashboard                    (calls /predict, displays results)
```

By step 7, your entire "AI brain" works from a Python script alone — you can test it without any web server. Steps 8–9 are just plumbing at that point.

4. Detailed Flow, Stage by Stage

Stage A — Data Engineering: Ingestion & Validation

```
Farmer uploads image
        │
        ▼
FastAPI receives file → saves to Object Storage (MinIO/S3)
        │
        ▼
Metadata extraction (GPS, timestamp, device info, user_id)
        │
        ▼
Validation: file type check, duplicate check, size check
```

Tech: FastAPI, MinIO or AWS S3, Pillow/OpenCV for basic file checks.

Stage B — Data Engineering: OpenCV Pipeline

```
Raw Image
   │
   ├── Blur Detection        → cv2.Laplacian(gray, CV_64F).var()
   ├── Brightness Check       → mean pixel intensity in HSV
   ├── Leaf Detection          → HSV thresholding + contours
   ├── Background Removal      → largest contour mask
   ├── CLAHE Enhancement       → cv2.createCLAHE()
   └── Resize to 224×224
   │
   ▼
Processed Image + quality_score + blur flag + leaf_detected flag
```

If quality checks fail (too blurry, no leaf detected), the pipeline should short-circuit and ask the farmer to re-upload **before** any model runs — this saves compute and is a nice thing to show judges as a UX detail.

Tech: OpenCV (`cv2`), NumPy.

Stage C — Data Science: Model Training (done offline, before deployment)

```
Raw Dataset (e.g. PlantVillage or your own collected images)
        │
        ▼
EDA — class balance, image sizes, corrupted files
        │
        ▼
Data Cleaning — remove corrupted/duplicate images
        │
        ▼
Train / Val / Test Split (e.g. 70/15/15, stratified by class)
        │
        ▼
Augmentation — Albumentations (rotation, flip, color jitter, cutout)
        │
        ▼
Transfer Learning — EfficientNet-B0 pretrained on ImageNet, fine-tune head
        │
        ▼
Fine-Tuning — unfreeze later layers, lower LR
        │
        ▼
Evaluation — accuracy, F1, confusion matrix, Grad-CAM sanity checks
        │
        ▼
Best Model → saved as .pth, versioned (e.g. crop_identifier_v1.pth)
        │
```

```
        ▼
Model Registry (can be as simple as a versioned folder + a JSON manifest for a hackathon)
```

This is exactly what your four notebooks map to: - `01_data_preparation.ipynb` → EDA, cleaning, split, augmentation setup - `02_train_crop_identifier.ipynb` → transfer learning + fine-tuning - `03_evaluation.ipynb` → metrics, confusion matrix, Grad-CAM - `04_inference.ipynb` → loading the saved model and running predictions — this is what you'll port into `services/crop_identifier/predictor.py`

Tech: PyTorch, `timm` (for EfficientNet weights), Albumentations, scikit-learn (metrics), matplotlib, `pytorch-grad-cam`.

Stage D — AI Inference Layer (runtime, once trained models exist)

```
Processed Image
      │
      ▼
Crop Identifier (EfficientNet-B0)  → crop="Tomato", confidence=0.99
      │
      ▼
Disease Classifier (conditioned on crop) → disease="Early Blight", confidence=0.97
      │
      ▼
Severity Estimator (OpenCV segmentation) → severity=32%, affected_area=18.4%
      │
      ▼
Pest Detector (YOLOv8, optional)   → bounding boxes if pests present
```

Note the disease classifier takes the crop as an input — this is why the shared `request` object matters; each stage reads what the previous stage wrote.

Stage E — Recommendation Layer

```
crop + disease + severity + weather + location + crop history
        │
        ▼
Prompt construction
        │
        ▼
LLM API call (Gemini / Llama 3 / Groq)
        │
        ▼
Structured recommendation: fertilizer, pesticide, irrigation, prevention tips
```

Tech: Any hosted LLM API + a weather API (OpenWeatherMap is a common free option).

Stage F — Orchestration & API

```python
# pipeline.py — no AI logic here, only sequencing
def run_pipeline(request):
    request = preprocess(request)          # OpenCV
    request = identify_crop(request)       # EfficientNet
    request = classify_disease(request)    # EfficientNet
    request = estimate_severity(request)   # OpenCV + AI
    request = generate_recommendation(request)  # LLM
    return request
```

```python
# main.py — FastAPI is a thin wrapper
@app.post("/predict")
def predict(file: UploadFile):
    request = create_request(file)
    result = run_pipeline(request)
    save_to_db(result)
    return result
```

Stage G — Persistence

```
users            (user details, farm details, crop history)
images           (original + processed paths, metadata)
predictions      (crop, disease, confidence, severity, timestamp)
recommendations  (fertilizer, pesticide, irrigation, prevention tips)
feedback         (farmer feedback on prediction accuracy — feeds future retraining)
```

Tech: PostgreSQL for structured data, MinIO/S3 for image blobs.

Stage H — Frontend

```
React (Vite) → single API call to /predict → renders crop, disease, severity, recommendation
```

Because all the intelligence already lives behind one endpoint, this stage should be the fastest part of the build.

Stage I — MLOps Loop (mention this even if you don't fully build it — judges like seeing it)

```
New predictions + farmer feedback
        │
        ▼
Performance monitoring (accuracy over time, confidence drift)
        │
        ▼
Drift detection (are new images statistically different from training data?)
        │
        ▼
Retraining trigger → new dataset → retrain → new model version → redeploy
```

5. Recommended Folder Structure

```
Smart-Farming/
├── backend/
│   ├── main.py
│   ├── pipeline.py              ⭐ orchestrator — no AI logic
│   ├── config.py
│   ├── services/
│   │   ├── preprocessing/       (OpenCV)
│   │   ├── crop_identifier/     (predictor.py)
│   │   ├── disease_classifier/  (predictor.py)
│   │   ├── severity/
│   │   ├── recommendation/      (LLM calls)
│   │   └── weather/
│   ├── models/                  (.pth weights, versioned)
│   ├── database/
│   └── utils/
├── frontend/                    (React/Vite)
├── datasets/
└── notebooks/                   (your 4 notebooks live here)
```

6. Full Tech Stack Reference

Layer	Tools
Data Engineering	Python, FastAPI, OpenCV, PostgreSQL, MinIO/S3, Pandas, NumPy
Data Science	PyTorch, timm, Albumentations, scikit-learn, matplotlib, Grad-CAM
AI Models	EfficientNet-B0/B2 (crop ID), EfficientNet-B2/ConvNeXt (disease), YOLOv8 (pests), optionally Segment Anything (leaf segmentation), Whisper (voice input), Llama 3/Gemini (recommendations)
Deployment	FastAPI, Docker, Nginx, React (Vite), Flutter (optional mobile)

7. Where to Make Changes in Your Existing Notebooks

Given what you've already built (`01_data_preparation.ipynb` → `04_inference.ipynb`), the concrete next

edits are:

1. **`01_data_preparation.ipynb`** — insert the OpenCV quality-check step (blur/brightness/leaf detection) as a filtering pass *before* your train/val/test split, so bad images never enter training.
2. **`02_train_crop_identifier.ipynb`** — confirm you're using `timm.create_model('efficientnet_b0', pretrained=True)` with a replaced classifier head sized to your crop classes, and that Albumentations transforms are applied in the `Dataset` / `DataLoader`, not just at load time.
3. **`03_evaluation.ipynb`** — add a Grad-CAM visualization cell if you don't have one yet; judges respond well to seeing *why* the model predicted a class, not just accuracy numbers.
4. **`04_inference.ipynb`** — this is the one to refactor into `backend/services/crop_identifier/predictor.py` as a single `predict_crop(image) -> (crop, confidence)` function, matching the shared-contract pattern described above.

Everything downstream (disease classifier, severity, recommendation, orchestrator, API) is new work layered on top of what these four notebooks already give you.